

Assignment 3. Cryptography

Project Release: May 10, 2026 KST

Deadline: May 22, 2026 (FRI) by 8:59:59 PM KST

The code and other answers you submit must be entirely your own work. You may discuss this assignment in small groups with classmates, but you must write your code and solutions yourself. Please acknowledge any collaborators you discussed the assignment with and cite any references you used. You may consult published references, provided that you cite them appropriately (e.g., in program comments).

Solutions must be submitted via Blackboard.

Introduction

This project will introduce you to control-flow hijacking vulnerabilities in application software. We will provide a series of vulnerable programs and a virtual machine environment in which you will develop exploits.

target0–target6 are required. target7 is extra credit.

Read this First

This project asks you to develop attacks and test them in a virtual machine you control. Attempting the same kinds of attacks against others' systems without authorization is prohibited by law and university policies and may result in *finer, expulsion, and jail time*. **You must not attack anyone else's system without authorization!** Per the course ethics policy, you are required to respect the privacy and property rights of others at all times, *or else you will fail the course*.

Setup

Buffer-overflow exploitation depends on specific details of the target system, so we are providing an Ubuntu VM in which you should develop and test your attacks. We'll use this precise configuration to grade your submissions, so you must not use your own VM.

1. Use the same VM provided for Assignment 2.
2. Inside the VM, install required packages:

```
sudo apt install libc6-dev-i386 gdb
```

3. Disable ASLR as below (cat should print 0.):

```
echo "kernel.randomize_va_space = 0" | sudo tee /etc/sysctl.d/01-disable-aslr.conf
sudo sysctl -p /etc/sysctl.d/01-disable-aslr.conf
cat /proc/sys/kernel/randomize_va_space
```

4. Download the zip file `pa3.tar.gz` from Blackboard. This file contains all of the programs you will exploit.
5. Decompress `pa3.tar.gz` with `tar xf pa3.tar.gz`. You should see a `targets` directory – enter that directory.
6. Run `./build.sh`. It will prompt you for usernames. Use your full English name without space. Running `./build.sh` should generate a cookie file. Take a look at its contents.

```
cse467@cse467:~/targets$ cat cookie
minkyungpark
27440
```

7. If you need to recompile your targets, you will need to run `./build.sh clean` before you run `./build.sh` again.

Resources and Guidelines

No Attack Tools! You must not use special-purpose tools meant for testing security or exploiting vulnerabilities. You must complete the project using only general purpose tools, such as `gdb`.

Control Hijacking Before you begin this project, review the slides from the buffer overflow lectures and attend discussion for additional details. You can find more details in the article "Smashing the Stack for Fun and Profit" available at <https://phrack.org/issues/49/14#article>.

GDB You will make extensive use of the GDB debugger. Useful commands are “`disassemble`”, “`info reg`”, “`x`”, and setting breakpoints. See the seminar tutorial!

x86 Assembly These are many good references for Intel assembly language, but note that our project uses the 32-bit x86 ISA with AT&T syntax. The stack is organized differently in x86 and x86_64. If you are reading any online documentation, ensure that it is based on the x86 architecture, not x86_64. Here is one reference to the syntax that we are using:

<https://www.ibiblio.org/gferg/ldp/GCC-Inline-Assembly-HOWTO.html#s3>

If you are getting a segfault A segfault means that you’re either jumping execution to or dereferencing an address that is incorrect. This means you’re probably on the right track because you’ve overwritten something and changed the program’s behavior! If you are stuck as to where to start looking, try to identify the addresses the exploit has changed and work from there, i.e. make sure the addresses you intended to change have actually been changed and nothing else.

Warning: Do not try to create a solution that depends on you manually setting environment variables. You cannot assume that the autograder will run your solution with the same environment variables that you have set.

Targets

The provided programs for this project are simple, short C programs with (mostly) clear security vulnerabilities. We are going to refer to these vulnerable programs as “targets”. We have provided source code and a build script that compiles all the targets. Your solutions must work against these targets as compiled and executed within the provided VM.

target0: Overwriting a variable on the stack (1pts)

This program takes input from `stdin` and prints a message. Your job is to provide input that makes it output: “Hi *{your name}*! Your grade is A+.”. To accomplish this, your input will need to overwrite another variable stored on the stack.

Here’s one approach you might take:

1. Examine `target0.c`. Where is the buffer overflow?
2. Start the debugger (`gdb target0`) and disassemble `_main`: (`gdb`) `disas _main`
Identify the function calls and the arguments passed to them.
3. Draw a picture of the stack. How are `name[]` and `grade[]` stored relative to each other?
4. How could a value read into `name[]` affect the value contained in `grade[]`? Test your hypothesis by running `./target0` on the command line with different inputs.

What to submit Create a Python program named `sol0.py` that prints a line to be passed as input to the target. Test your program with the command line:

```
python3 sol0.py | ./target0
```

Hint: In Python 3, you should work with bytes rather than Unicode strings. To construct a byte literal, use this syntax: `b“\xnn”`, where `nn` is a 2-digit hex value (e.g., `b“\x70”`). To repeat a byte `m` times, you can do: `b“\xnn * m”`. To output a sequence of bytes, use:

```
import sys
sys.stdout.buffer.write(b“\x61\x62\x63”)
```

Don’t use `print()`, because it automatically encodes whatever is being printed with the default encoding of the console. We don’t want our payload to be encoded, so we use `sys.stdout.buffer.write()`.

target1: Overwriting the return address (2pts)

This program takes input from `stdin` and prints a message. Your job is to provide input that makes it output: “Your grade is perfect.” Your input will need to overwrite the return address so that the function `vulnerable()` transfers control to `print_good_grade()` when it returns.

1. Examine `target1.c`. Where is the buffer overflow?
2. Disassemble `print_good_grade`. What is its starting address?

3. Set a breakpoint at the beginning of `vulnerable` and run the program.
`(gdb) break vulnerable`
`(gdb) run`
4. Disassemble `vulnerable` and draw the stack. Where is `input[]` stored relative to `%ebp`? How long an input would overwrite this value and the return address?
5. Examine the `%esp` and `%ebp` registers:
`(gdb) info reg`
6. What are the current values of the saved frame pointer and return address from the stack frame? You can examine two words of memory at `%ebp` using:
`(gdb) x/2wx $ebp`
7. What should these values be in order to redirect control to the desired function?

What to submit Create a Python program named `sol1.py` that prints a line to be passed as input to the target. Test your program with the command line:

```
python3 sol1.py | ./target1
```

When debugging your program, it may be helpful to view a hex dump of the output. Try this:

```
python3 sol1.py | hd
```

Remember that x86 is little endian. Use Python's `struct` module or `to_bytes` method to output little-endian values:

```
from struct import pack
sys.stdout.buffer.write(pack("<I", 0xDEADBEEF))

import sys
sys.stdout.buffer.write(0xDEADBEEF.to_bytes(4, "little"))
```

target2: Redirecting control to shellcode (2pts)

The remaining targets are owned by the `root` user and have the `suid` bit set. Your goal is to cause them to launch a shell, which will therefore have root privileges. This and later targets all take input as command-line arguments rather than from `stdin`. Unless otherwise noted, you should use the shellcode we have provided in `shellcode.py`. Successfully placing this shellcode in memory and setting the instruction pointer to the beginning of the shellcode (e.g., by returning or jumping to it) will open a shell.

1. Examine `target2.c`. Where is the buffer overflow?
2. Create a Python program named `sol2.py` that outputs the provided shellcode:

```
from shellcode import shellcode
import sys
sys.stdout.buffer.write(shellcode)
```

3. Set up the target in GDB using the output of your program as its argument:

```
gdb --args ./target2 "$(python3 sol2.py)"
```

4. Set a breakpoint in **vulnerable** and start the target.
5. Disassemble **vulnerable**. Where does **buf** begin relative to **%ebp**? What's the current value of **%ebp**? What will be the starting address of the shellcode?
6. Identify the address after the call to **strcpy** and set a breakpoint there:

```
(gdb) break *addr
```


Continue the program until it reaches that breakpoint.

```
(gdb) cont
```
7. Examine the bytes of memory where you think the shellcode is to confirm your calculation:

```
(gdb) x/32bx 0xaddress
```
8. Disassemble the shellcode:

```
(gdb) disas/r 0xaddress,+32
```


How does it work?
9. Modify your solution to overwrite the return address and cause it to jump to the beginning of the shellcode.

What to submit Create a Python program named **sol2.py** that prints a line to be used as the command-line argument to the target. Test your program with the command line:

```
./target2 $(python3 sol2.py)
```

If you are successful, you will see a root shell prompt (**#**). Running **whoami** will output **"root"**. Running **exit** will return to your normal shell.

If your program segfaults, you can examine the state at the time of the crash using GDB with the core dump: **gdb ./target2 core**. The file **core** won't be created if a file with the same name already exists. Also, since the target runs as root, you will need to run it using **sudo ./target2** in order for the core dump to be created.

target3: Overwriting the return address indirectly (3pts)

In this target, the programmer is using a safer function (**strncpy**) to copy the input string to a buffer. Therefore, the buffer overflow exploit is restricted and cannot directly overwrite the return address. However, this programmer has miscalculated the length of the buffer. Hopefully this will help you to find another way to gain control. Your input should cause the provided shellcode to execute and open a root shell.

What to submit Create a Python program named **sol3.py** that prints a line to be used as the command-line argument to the target. Test your program with the command line:

```
./target3 $(python3 sol3.py)
```

target4: Beyond strings (3pts)

This target takes as its command-line argument the name of a data file it will read. The file format is a 32-bit count followed by that many 32-bit integers. Create a data file that causes the provided shellcode to execute and opens a root shell.

Hint: First figure out how an attacker can cause a buffer overflow in this program. Note that the `read_elements` function breaks the for-loop once the end of the file is reached, so the 32-bit count does not need to be truthful.

What to submit Create a Python program named `sol4.py` that outputs the contents of a data file to be read by the target. Test your program with the command line:

```
python3 sol4.py > tmp; ./target4 tmp
```

target5: Bypassing DEP (3pts)

This program resembles `target2`, but it has been compiled with data execution prevention (DEP) enabled. DEP means that the processor will refuse to execute instructions stored on the stack. You can overflow the stack and modify values like the return address, but you can't jump to any shellcode you inject. You need to find another way to run the command `/bin/sh` and open a root shell.

What to submit Create a Python program named `sol5.py` that prints a line to be used as the command-line argument to the target. Test your program with the command line:

```
./target5 $(python3 sol5.py)
```

For this target, it's acceptable if the program segfaults after the root shell is closed.

target6: Variable stack position (3pts)

When we constructed the previous targets, we ensured that the stack would be in the same position every time the vulnerable function was called, but this is often not the case in real targets. In fact, a defense called ASLR (address-space layout randomization) makes buffer overflows harder to exploit by changing the position of the stack and other memory areas on each execution. This target resembles `target2`, but the stack position is randomly offset by 0–255 bytes each time it runs. You need to construct an input that always opens a root shell despite this randomization.

What to submit Create a Python program named `sol6.py` that prints a line to be used as the command-line argument to the target. **Your solution must not cause the program to print out any error messages.** Test your program with the command line:

```
./target6 $(python3 sol6.py)
```

target7: Return-oriented programming (Extra Credit)

This target is identical to target2, but it is compiled with DEP enabled. Implement a ROP-based attack to bypass DEP and open a root shell.

It will be helpful to use a tool such as ROPgadget; this is an exception to the “no attack tools” policy. You can download and install ROPgadget by using:

```
sudo apt install python3-pip
sudo pip3 install ropgadget
```

View its usage by running ROPgadget -h. The --binary, --badbytes, --multibr, and --ropchain flags will be particularly helpful.

1. Though there are a number of ways you could implement a ROP exploit, for this target you should use the `setuid` syscall to become root, followed by the `execve` syscall to run the `/bin/sh` binary. This is equivalent to:

```
setuid(0);
execve("/bin/sh", 0, 0);
```

2. For an extra push in the right direction, `int 0x80` is the assembly instruction for interrupting execution with a syscall. If the `EAX` register contains the number 23, the syscall will be `setuid`; if it contains 11, the syscall will be `execve`. You need to figure out what values you need for `EBX`, `ECX`, and `EDX`, and set them using ROP gadgets!
3. We recommend that you start by getting the `execve` call to work on its own, without `setuid`. When you do this correctly, it will open a shell, but you won't be root. Then modify your solution to make it call `setuid` first, and you'll get a root shell.

What to submit Create a Python 3 program named `sol7.py` that prints a line to be used as the command-line argument to the target. Test your program with the command line:

```
./target7 "$(python3 sol7.py)"
```

For this target, it's acceptable if the program segfaults after the root shell is closed.

Note: Tutors/TAs will not offer help for the extra credit problem.

Submission Checklist

The `binary.zip` file is a ZIP file, or any other compressed archive, containing the binary files target0 through target6 (or target7), just in case. These binary files will not be used for grading.

- `cookie` [Generated by `./build.sh` based on your name.]
- `sol [0 - 6].py`
- `sol7.py` (only needed for EC)
- `binary.zip`

Your files can make use of standard Python libraries and the provided **shellcode.py**, but they must be otherwise self-contained. Do not submit any additional files. Be sure to test your solutions on an unmodified copy of the provided VM, without installing or updating any packages, changing any environment variables, or making any changes other than those specified in the instructions.